



29. September 2026

Übungen zur Vorlesung Software Engineering I WS 2026 / 2027

Übungsblatt Nr. 1

(Abgabe bis: Mittwoch, den 7. Oktober 2026, 09:00 Uhr)

Aufgabe 1 (Wiederholung Java, Erstes Muster, Blackbox-Testing, 20 Punkte):

In dieser Vorlesung werden Sie eine Reihe von neuen Methoden zur Entwicklung von komplexen Software-Systemen kennenlernen. Ein solides Grundwissen aus den Bereichen Objektorientierung sowie Programmierung in Java ist dabei eine unabdingbare Voraussetzung für das Erlernen sowie für das Verständnis dieser Methoden! Daher werden in den Übungen Kenntnisse aus diesen Gebieten als Vorbereitung zu den Kapiteln 6 (Entwurfsmuster) und 7 (Entwicklung mit JUnit) wiederholt bzw. intensiviert sowie erste Themengebiete daraus adressiert.

Es seien zwei Java-Klassen `Client` und `GermanTranslator` sowie das Interface `Translator` gegeben, welches sie auf dem „Code-Repository“ der Vorlesung herunterladen können:

<https://git.fslab.de/salda2m/se1ws26>

Weitere Infos zur Verwendung eines GitLab-Repository erhalten Sie in Kürze.

Ihre konkreten Aufgaben:

1.)

Es soll Ihre Aufgabe sein, eine Kommunikationsverbindung zu programmieren, so dass ein Objekt der Klasse `Client` zur Laufzeit die Methode `translateNumber( int zahl )` *direkt* von einem Objekt der Klasse `GermanTranslator` aufrufen kann. Dieser Aufruf sollte in dem konkreten Beispiel aus der Methode `display()` erfolgen. Die Bedingung ist dabei, dass in der Klasse `Client` keine Instanz von der Klasse `GermanTranslator` oder auch von einer anderen Klasse mit dem Befehl **new** unmittelbar erzeugt werden darf! Zudem soll in der Klasse `Client` *streng (d.h. nur)* gegen das Interface `Translator` implementiert werden.

- Wie kann diese Kommunikationsverbindung nun dennoch mit Hilfe einer *zusätzlichen* „Fabrik“-Klasse, welche die dazu notwendige Objekt-Erzeugung übernimmt, aufgebaut werden? In welchem Package sollte diese zusätzliche Klasse liegen? Bitte beachten Sie dabei auch die Hinweise bzw. Anforderungen aus den Kommentaren

der Klassen, Methoden und des Interfaces. So sollte z.B. auch das Datum der Erzeugung eines Translator-Objekts (siehe setter-Methode) gesetzt werden.

- Welches Entwurfsmuster (engl.: *design pattern*) könnte für die Problematik der Objekt-Erzeugung verwendet werden? Was ist der software-technische Nutzen bei der Verwendung des Entwurfsmusters? Gratistipp: Hinweise für das korrekte Pattern finden Sie bei unten angegebenen Video-Tutorien ;-)
- Wie muss man den Source Code des Interface ggf. anpassen, um mögliche auftretende Kompilierfehler zu beseitigen?

2.)

Geben Sie der Methode `translateNumber( int zahl )` eine Implementierung gemäß folgender Spezifikation:

- Für die *gültigen* Zahlen 1 bis 100 soll die schriftliche Schreibweise zurückgeliefert werden. Beispiel: 1 → eins; 2 → zwei 67 → siebenundsechzig ... usw.
- Falls eine *ungültige* Zahl größer 100 oder kleiner 1 übergeben wird, dann soll eine entsprechende Fehlernachricht („Übersetzung der Zahl [*übergebeneZahl*] nicht möglich ([*Ausgabe der Versionsnummer des Translators aus Interface*])“) als Rückgabewert der Methode zurückgegeben werden.

Schauen Sie sich bei der Entwicklung eines passenden Algorithmus zur Übersetzung die sprachliche Struktur von Zahlen an (insbesondere Zahlen > 20 (Beispiel: „drei-und-zwanzig“). Für zukünftige Zwecke sollte auch der maximale INT-Wert übersetzt werden.

3.)

Testen sie ihre Implementierung der Methode `translateNumer` aus der Klasse `GermanTranslator` mit einem **Blackbox**-Test. Über diese Test-Art sollten sie sich selber informieren, siehe dazu auch die Quelle (Spillner und Linz, 2012) auf dem Literatur-Repository. Die Tests müssen in einer *separaten* (Test-)Klasse innerhalb eines eigenen Test-Packages platziert werden. Entwickeln sie eine repräsentative Menge an Positiv- und Negativtestfällen auf Basis geeigneter **Äquivalenzklassen**, um die Klasse *hinreichend* zu testen. Für die Implementierung des Blackbox-Tests können sie das Framework JUnit5 gerne verwenden, eine erste Vorlage finden sie auf GitHub (Test-Klasse „`GermanTranslatorTest`“). Definieren Sie zudem passende fachliche **Grenzwerte**, die es gilt zu testen. Entwickeln Sie auf dieser methodischen Basis mindestens 10 Testfälle. Bilden Sie die Testfälle auf einzelne Testmethoden sinnvoll ab. Testen Sie auch, ob das Datum in `GermanTranslator` korrekt gesetzt wurde.

Zur Spezifikation der Testfälle und der Äquivalenzklassen sollten Sie die Excel-Vorlage „TemplateTestCase v1.6“ / Blatt „Simple Test Suite“ verwenden (siehe Übung Nr. 1).

Bitte beantworten Sie auch folgende Fragen:

- Was ist der Vorteil einer separaten Test-Klasse?
- Was ist bei einem Blackbox-Test der Sinn von Äquivalenzklassen?
- Warum ist ein Blackbox-Test mit JUnit auf der Klasse `Client` nicht unmittelbar durchführbar?

Generelle Anmerkungen zur Abgabe:

Für all diese Teilaufgaben sollten sie die Source Codes als Lösung bereitstellen. Selbstverständlich können sich die Sources Codes aller Teilaufgaben in einem Projekt befinden, das zusammen gezippt auf LEA oder aber auf einem öffentlichen GitHub- oder GitLab-Repository bereitgestellt werden kann. Bei der Bereitstellung auf einem GitHub-/GitLab-Repository bitte den HTTPS-Link dazu kopieren und auf LEA in einer Text-Datei hochladen (weitere Hinweise in den u.g. Video-Tutorien).

Die hier gestellten Fragen sollten sie kurz in einer separaten Text-Datei beantworten, bitte daraus ein PDF erzeugen und auf LEA hochladen. Eine Beantwortung der Fragen in einer readme.md innerhalb eines GitHub-Repository ist auch möglich. Das Excel direkt hochladen oder auch hier eine PDF-Datei generieren aus dem Blatt.

Empfehlenswerte Literatur für den Black Box Test:

Spillner, A. und Linz, T.: Basiswissen für den Softwaretest. dpunkt.Verlag, 5. Auflage. 2012 (Kapitel 5, siehe „Software Engineering Literatur-Repro“):

<https://github.com/aldaGit/se-repository>

Aufgabe 2 (Modellierung mit UML-Klassendiagrammen, 10 Punkte)

Modellieren sie ihr finales Software-System aus der Aufgabe 1-1 (inkl. Test-Klasse) als *einfaches* UML-Klassendiagramm! Für diese, aber auch für die nächsten Übungen rund um die Modellierung von Software empfehle ich Ihnen folgendes Tool, das Sie sich im Rahmen der ersten Woche anschauen sollten:

- Draw.io / Diagrams

Schlankes browser-basiertes Tool, keine Installation auf ihrem Rechner notwendig! Abspeicherung der Dokumente in verschiedenen Formen möglich (Lokal, Cloud). Läuft nativ ohne Plugin auf allen gängigen Browsern. Ein Export ist z.B. als PDF möglich.

Quelle: <https://app.diagrams.net/>

Als ersten Einstieg in die Welt der Sprache UML empfehle ich folgendes Werk:

Rupp, Chris: UML 2 Glasklar, 4. Aufl., 2012 (online verfügbar in der Bibliothek). Eine gute Einführung zu Klassendiagrammen finden sie in diesem Buch in Kapitel 6.

Entwickeln Sie ein Klassendiagramm auf Basis von Draw.io. Bitte das Modell in ein PDF-Dokument umwandeln und dann auf LEA hochladen.

Weitere Hinweise für die anstehenden Übungen:

Verwendung der Entwicklungsumgebung (IDE)

In den Vorlesungen SE-1 und SE-2 wird die Verwendung der Entwicklungsumgebung IntelliJ empfohlen, die IDE hat sich in den letzten Jahren als Standard etabliert. Hier empfiehlt sich der Download der **Ultimate-Version**, die als registrierter Student kostenlos bezogen werden kann! Unübertroffen ist die Auto Completion Funktion, welche IntelliJ recht populär und beliebt gemacht hat. Auch die KI-Unterstützung durch diverse CoPilot-Erweiterungen ist professionell. In den Übungen wird IntelliJ für Demo-Zwecke verwendet. Auch die Verwendung eines GitHub- / GitLab-Repository ist hier sehr intuitiv realisiert worden.

<https://www.jetbrains.com/idea/>

Verwendung von GitLab

Als Einführung in IntelliJ finden Sie auf LEA ein von uns produziertes, mehrteiliges Video-Tutorial, welches die Installation des Tools, das Aufsetzen eines Projekts mit JUnit5 sowie die Integration von Git und GitHub / GitLab verdeutlicht. Die Links finden sie auch in LEA (Reiter: Nützliche Videos für Übungen) sowie in der readme.md auf dem Code-Repository. Hinweis: in diesen Videos wird auf ein Repository codesSE2020 verwiesen. Dieses Jahr verwenden wir natürlich das neue Repository se1ws26.

Generelle Hinweise zu den Übungsaufgaben im Software-Engineering

Es kann in dieser Übung (und auch in anderen Übungen im Software-Engineering) durchaus sein, dass es aufgrund komplexer Aufgabenstellungen bei einzelnen Teilaufgaben unterschiedliche Lösungen existieren, die bei der Bewertung bzw. bei der Besprechung als korrekt angenommen werden können. Die Musterlösungen repräsentieren dabei stets eine mögliche Variante, von der es durchaus Abweichungen in ihren Lösungen geben kann. Somit haben Sie z.B. bei der Benennung von Variablen, Methodennamen oder generellen Entwurfsentscheidungen etc. gewisse Freiheitsgrade.